CHAPTER 20: DOCUMENTATION THAT SURVIVES YOUR ABSENCE

Summary: *Documentation gets judged by the wrong test. People ask whether it is complete, or accurate, or nicely formatted. The test that matters is whether somebody who was never involved can do their job from it after everyone who built the thing has gone. This chapter names the three readers a document has to serve, what makes a decision record hold up under audit, and why a route for asking questions is not the same as an answer.*

The Three Readers

Any serious document about a system serves three people, and each needs something the others do not.

The **implementer** has to build it. What they need is what to build: the exact contracts, the interfaces, the settings, the behaviour under each condition.

The **operator** has to run it after it exists, and diagnose it when it misbehaves. What they need is procedure: how to check whether it is healthy, what to do when it is not, what each failure looks like.

The **reviewer** has to establish, later and possibly under pressure, why it was built this way. What they need is reasoning: the criteria, the constraints, and what was rejected.

Name it: the **three readers** are the audiences any document has to survive, and most documents serve one and quietly assume the other two will ask somebody.

How deep the document goes is decided by what its **readers have to make**. That single sentence settles most items in this objective. Write for a group who will first construct the thing and then **keep it running**, and you owe them both halves: the precise interfaces they code against, and the routines they will be executing every week once it is live. Choosing one is choosing which of their two jobs to leave undocumented.

Sufficiency Means No Remaining Questions, Not a Way to Ask Them

The sharpest distinction here, and the one that produces the most attractive wrong answer.

A package is **sufficient** when a reader can tell from it what has to be **built**, and can also tell why the **design came out that way**. Two separate things, and each removes questions the implementer would otherwise carry to somebody.

The near miss is any option that **supplies a route for asking**: a named contact, an office-hours slot, a channel to post in, a weekly call with the architect. Every one of those is helpful. **None of them is documentation**, because each one converts a **written answer** into a **dependency on a person being available**.

That dependency has a specific failure mode and it is the reason this objective exists. The person leaves, or is busy, or misremembers, or gives a different answer to the next person who asks. A **question that has to be asked has not been answered**.

The **named anti-pattern** is the **handover that names a person** instead of **stating a fact**. It reads as generous and it is the thing that stops the handover from being a handover.

What Makes a Decision Record Hold Up

Four properties, and the fourth is the one people leave out.

Written. A decision that lives in a conversation, however clear at the time, does not exist for anyone who was not in it.

Dated. Because the question is never simply “why this?” It is “why this, given what was known then?”

Attributed. So a reader knows whose judgement it was and can weigh it, or ask them.

Carrying the rejected alternatives. Auditability means rebuilding how somebody was thinking on the **day they decided**, rather than simply learning which way it went.

That fourth property is what separates a record from an announcement. Knowing the team chose approach B tells a future reader almost nothing. Knowing they chose B over A and C, for stated reasons, under stated constraints, tells them whether the decision still holds now that one of those constraints has changed.

A **record of outcomes cannot be audited. A record of reasoning** can.

The artifact has a standard name and a standard shape. An **architecture decision record**, usually shortened to **ADR**, carries the **context** that made the decision necessary, the **decision** itself, the **alternatives considered**, the **consequences** accepted, and the **date and authors**. **One decision to a record**. Where a team already writes them, naming the practice is enough; where they do not, those five fields are what you are asking for.

Handoff Transfers Two Capabilities

When a system passes to another team, the handoff has to transfer two specific things, and naming them keeps you out of trouble.

A **way to find out whether a change broke anything**. Something the new team can run for themselves and read the result of. Without it they are frozen: any change is unbounded risk, so nothing gets changed, and the system rots in place while everybody calls it stable.

Why the **thing is shaped the way** it is. Why these components and not some other set. Without it the new team's first significant change fights a constraint they cannot see, and they will conclude the original design was stupid, which is the standard prelude to reintroducing a bug somebody already fixed.

Three things get offered instead, and each fails in a recognisable way:

Offered instead	Why it fails
A **dependency on people**	"Ask the original team" is not a transfer. It is the absence of one, formalised

Offered instead	Why it fails
Undistilled raw material	Ticket histories, chat logs, meeting recordings. The information may be in there; the answer is not, and finding it costs more than asking
A **fact that expires**	Current configuration values, present-day thresholds, this quarter's volumes. True today, misleading in six months, and nothing in the document says which

That last row is worth carrying. Anything you write down that will **silently become false** needs either a **mechanism to keep it current** or a **note saying when it was true**. A document that ages badly does more damage than one that never existed, because people trust it.

How Wrong Answers Are Built on Documentation Questions

- A **channel offered where an answer was required**. The test: does this **eliminate the question**, or **route** it?
- **Depth chosen for the writer's convenience rather than the reader's job**. The test: what do these **readers have to make**, and can they **make** it from this?
- An **outcome recorded without its reasoning**. The test: could a reader tell whether the decision **still holds under a changed constraint**?
- **Raw material offered as documentation**. The test: is the answer in here, or merely **somewhere** in here?
- A **fact with an expiry and no date**. The test: what in this **document will be false in six months**, and would **anyone know**?

When the scenario says	The answer is usually	The trap is usually
A package must be sufficient for implementers	A reader can tell what to build, and why the design came out that way	A named contact, office hours, or a channel for questions
The audience will first construct and then run the system	Both the interfaces they code against and the routines they will execute	Choosing one, which leaves half their job undocumented
A decision must survive an audit	Written, dated, attributed, with the rejected alternatives	Recording the outcome alone
A future reader must judge whether a decision still holds	The criteria and constraints at the time, plus what was rejected	A statement of what was chosen
A system is handed to another team, two things wanted	A way to find out whether a change broke anything, and why the thing is shaped as it is	Access to the original team, raw ticket history, or current settings
Ticket logs and chat history are offered	Distil them into answers	Handing over the archive and calling it documentation
Configuration values are documented	Note when they were true, or keep them current automatically	Writing today's numbers with no date

The habit worth carrying: read your own document as **somebody who joined last week** and **cannot ask anybody anything**. Everywhere you would have to guess is a defect, and every guess you would get wrong is the one that matters.

What to Remember

- **Three readers**: the **implementer** needs what to **build**, the **operator** needs how to **run and diagnose** it, and the **reviewer** needs why it was **decided this way**. Most documents serve one and assume the others will ask somebody.

- How deep a document goes is decided by what its **readers have to make**. Writing for a group who will **first construct the thing and then keep it running** means carrying **both the precise interfaces they code against** and the **routines they will execute once it is live**.

- A package is sufficient when a reader can tell from it what has to be **built** and why the **design came out that way**, both of which **remove questions**. A named contact, office hours or a channel supplies a **route for asking, which is not an answer**, and it converts written knowledge into a **dependency on somebody being available**.

- A decision record must be **written, dated, attributed**, and **carry the rejected alternatives**, because auditability means **rebuilding how somebody was thinking on the day they decided**, not simply learning which way it went. A record of **outcomes** cannot be audited; a record of **reasoning** can.

- A handoff transfers **two capabilities**: a **way to find out whether a change broke anything**, and an **account of why the thing is shaped the way** it is. Without the first, **nothing can safely be changed**. Without the second, the new team's first change **fights a constraint they cannot see**.

- **Three substitutes fail**: a **dependency on people**, **undistilled raw material** like ticket and chat history, and **facts that expire**. With raw material the **information may be present while the answer is not**, and an expiring fact **becomes quietly false while people still trust** it.

- Anything written down that will **silently become untrue** needs either a **mechanism keeping it current** or a **note saying when it was true**.